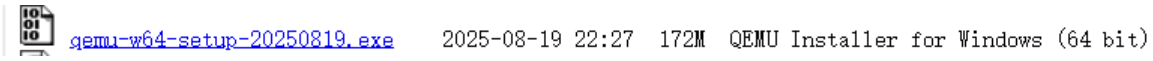
一、环境准备

1.在电脑上安装QEMU

可以参考这一篇CSDN上的文章windows下qemu的下载及安装_qemu下载地址-CSDN博客

windows 64bit QEMU安装包下载网址：https://qemu.weilnetz.de/w64/

我安装的是这个版本

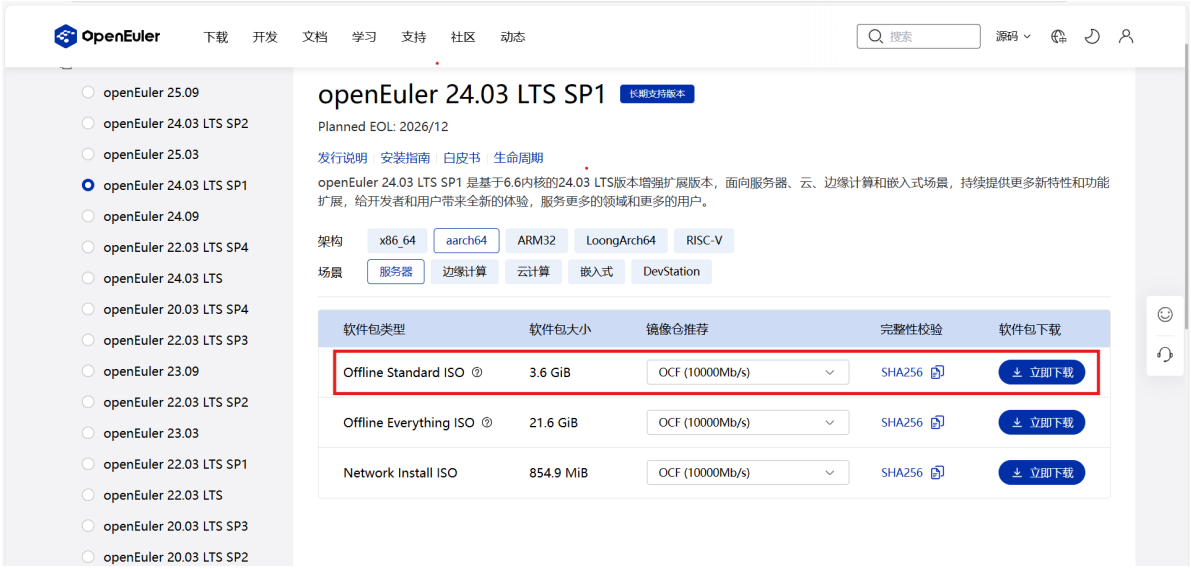


2.在电脑上安装OpenEuler系统（编译环境）

这一块我在csdn上没有找到很好的参考，所以这部分我自己来写一下。

首先我们先去openeuler官网上去下载这个24.03-LTS 的 `aarch64` ISO镜像（文件后缀为 `.iso`），也就是下面我圈出来的这个。

openEuler所有版本下载 | 下载中心 | openEuler社区



然后我们还要下载一个QEMU_EFI.fd文件。

http://releases.linaro.org/components/kernel/uefi-linaro/16.02/release/qemu64/QEMU_EFI.fd

这个链接点进去之后就应该是直接下载的。

下载好了以后，我们要把这两个文件全部都要移动到我们新建的一个文件夹当中，比如我这里就全部安装到了D盘下的qemu_vm文件夹下。（最上面那个文件先不管，后面装）



在这个文件夹下打开终端，或者你自己cd过来也行，输入这一串指令

```
qemu-img create -f qcow2 openEuler_vm.qcow2 20G
```

就是相当于在这个目录创建一个20GB的动态分配虚拟硬盘文件

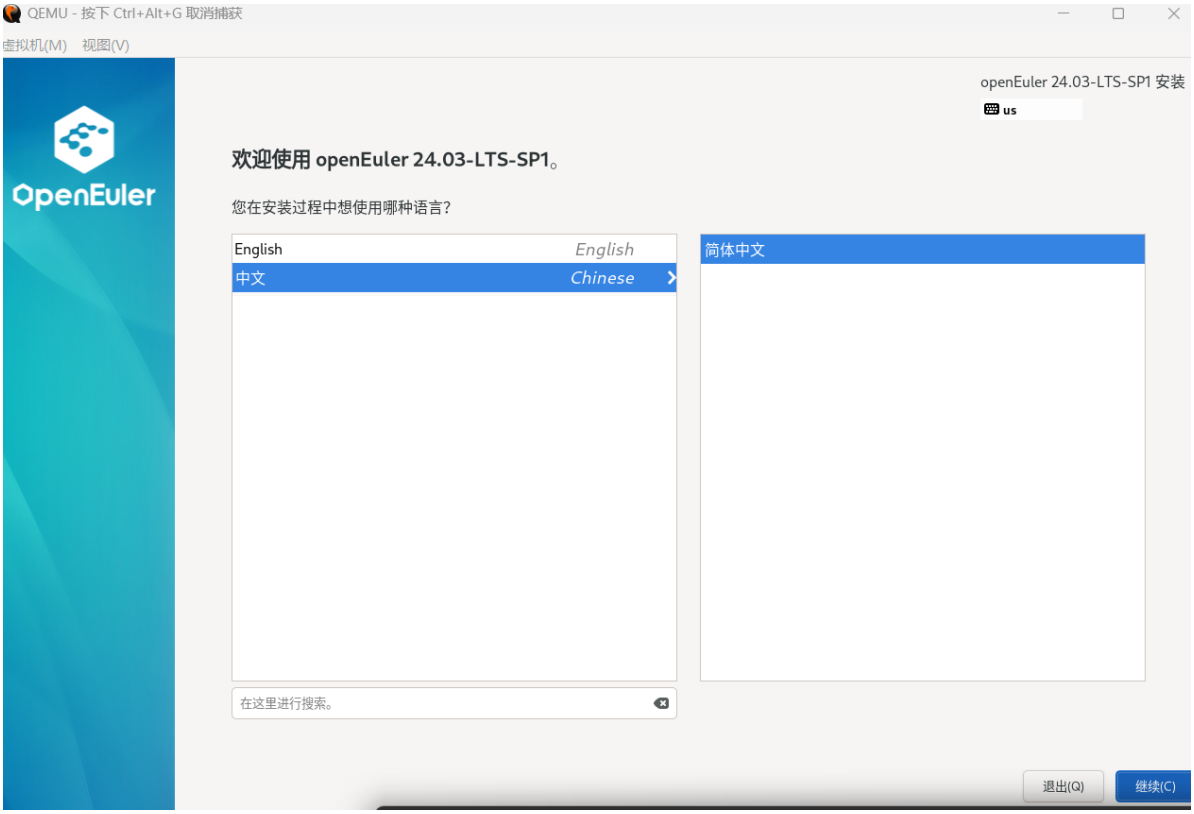
下一步，我们仍然是在刚才的终端中，输入这样的长命令

```
qemu-system-aarch64 `
-m 4096 -smp 4 `
-cpu cortex-a72 `
-M virt `
-bios D:\qemu_vm\QEMU_EFI.fd `
-device VGA `
-device usb-ehci -device usb-kbd -device usb-mouse `
-drive if=none,file=D:\qemu_vm\openEuler_vm.qcow2,format=qcow2,id=hd0 `
-device virtio-blk-device,drive=hd0 `
-cdrom D:\qemu_vm\openEuler-22.03-LTS-SP1-aarch64-dvd.iso `
-netdev user,id=net0 `
-device virtio-net-device,netdev=net0
```

```
qemu-system-aarch64: 这是QEMU的ARM64架构模拟器可执行文件。
-m 4096 虚拟机的内存大小为4GB。
-cpu cortex-a72: 指定模拟的CPU类型为Cortex-A72，一个ARMv8-A架构的处理器核心。
-M virt: 通用的虚拟机平台。
-bios "D:\qemu_vm\QEMU_EFI.fd": 指定EFI固件文件的路径，用于引导使用UEFI的虚拟机操作系统。
-device VGA: 添加一个虚拟VGA设备，用于图形输出。
-device nec-usb-xhci: 添加一个NEC USB xHCI USB控制器。
-device usb-mouse: 添加一个虚拟USB鼠标设备。
-device usb-kbd: 添加一个虚拟USB键盘设备。
-drive if=none,file="D:\qemu_vm\testEuler1.qcow2",id=hd0: 定义虚拟硬盘镜像文件；
id=hd0分配ID为hd0。
-device virtio-blk-device,drive=hd0: 使用virtio虚拟设备接口将虚拟硬盘连接到虚拟机。
-drive if=none,file="D:\qemu_vm\openEuler-22.03-LTS-SP1-aarch64-
dvd.iso",id=cdrom,media=cdrom: 定义虚拟CD-ROM驱动器，指定openEuler的ISO镜像文件。
-device virtio-scsi-device: 添加一个virtio SCSI控制器。
-device scsi-cd,drive=cdrom: 将虚拟CD-ROM驱动器连接到SCSI控制器。
-net nic: 添加一个虚拟网卡到虚拟机。
-net tap,ifname=tap0,script=no,downscript=no: 配置虚拟网络接口，使用TAP设备tap0。
script=no和downscript=no阻止QEMU自动管理TAP设备的启动/停止状态。
```

目前来看内存为4GB貌似是有点卡，不知道为什么，内存多的可以多分配一点。

然后是这个样子的，直接**Install openEuler 24.03-LTS-SP1**回车



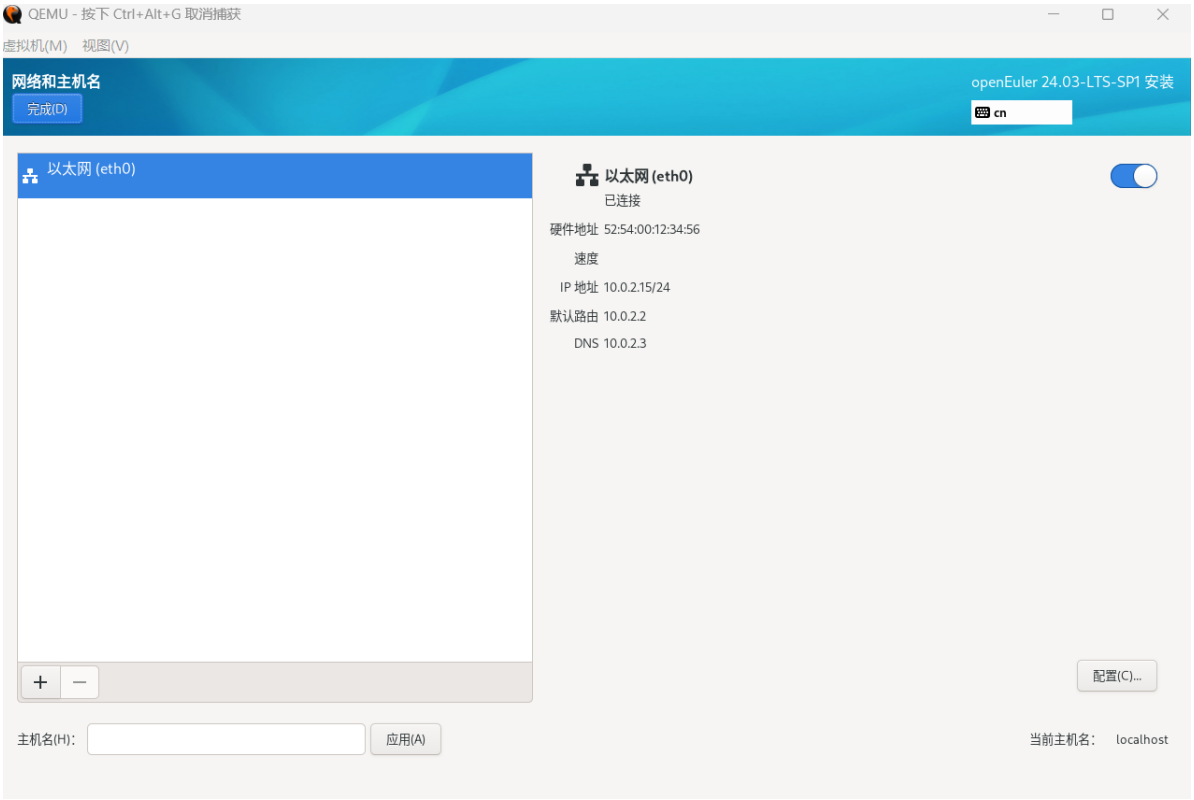
选中文，继续

安装目的地选择我们刚才分配的20GB的磁盘，完成





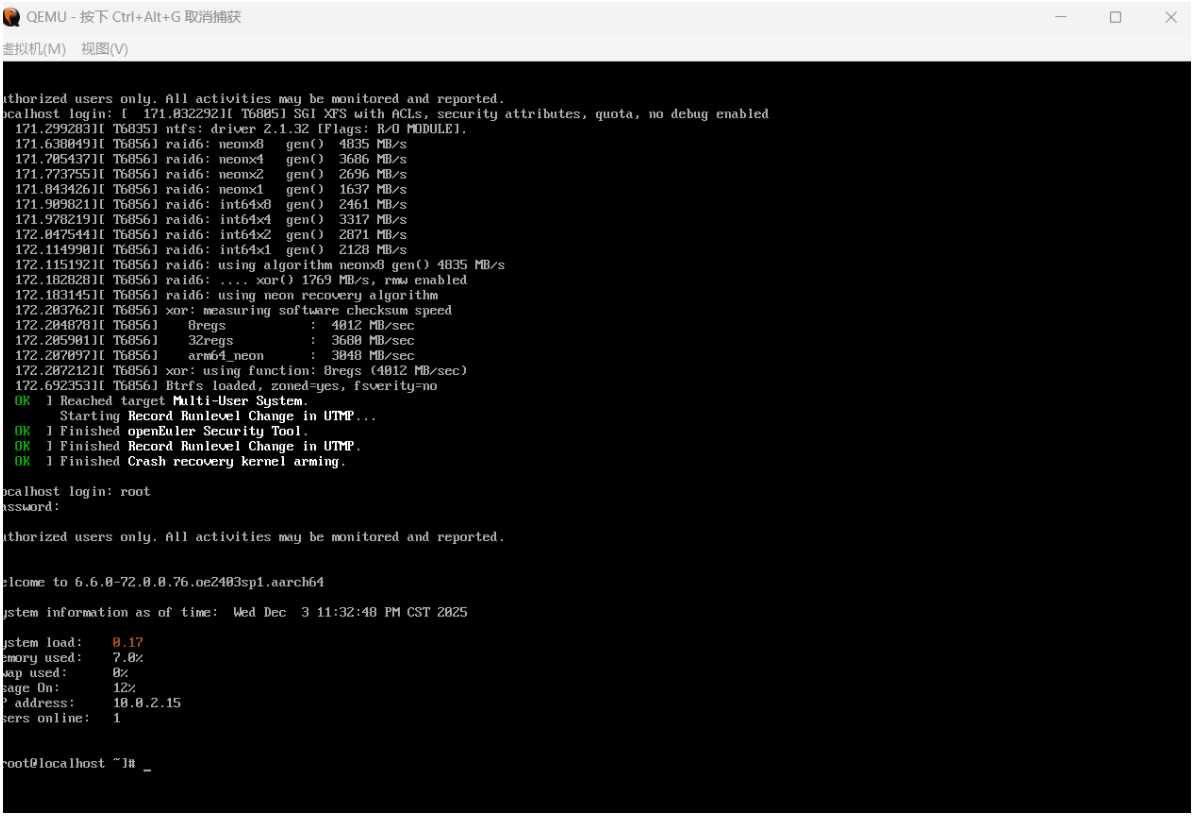
网络和主机名的以太网打开，完成



root账户，这里不建议输入数字上的特殊字符，好像是有点bug，很奇怪，也可以创建一个普通用户，点击开始安装



重启之后，差不多是这样的界面



你要输入账号和密码，如果你跟随着我上面的步骤来做的话，那账号就是root。

在这样的界面下，我们还要安装一些后续研究和编译UniProton内核所必需的工具链。

```
# 1. 更新系统软件包列表（确保能获取到最新版本的软件）这个时间最长，要三四个小时
sudo dnf update -y

# 2. 安装“开发工具”组，这是核心，包含了gcc, make, gdb, autoconf等
sudo dnf groupinstall "Development Tools" -y
```

```
# 3. 安装Git（用于克隆代码）、CMake（另一种构建工具）、Vim（编辑器）
sudo dnf install git cmake vim -y

# 4. 安装一些常用的依赖库
sudo dnf install ncurses-devel libuuid-devel -y

# 5. （可选但推荐）安装中国境内的dnf源镜像，后续安装软件会更快
sudo dnf install -y openeuler-repos
sudo dnf config-manager --set-enabled everything
```

执行完成后，用以下命令验证关键工具是否安装成功：

```
gcc --version
make --version
git --version
```

然后我们来获取UniProton内核源码

```
cd ~
git clone https://gitee.com/openeuler/UniProton.git
```

并且进入源码目录

```
cd UniProton
ls -la
```

至此全部完成。

安装完成后，你下次启动时**必须去掉** `-cdrom` **参数**，否则又会进入安装界面。你的启动命令将变成（注意第8行已被移除）：

```
qemu-system-aarch64 `
-m 4096 -smp 4 `
-cpu cortex-a72 `
-M virt `
-bios D:\qemu_vm\QEMU_EFI.fd `
-device VGA `
-device usb-ehci -device usb-kbd -device usb-mouse `
-drive if=none,file=D:\qemu_vm\openEuler_vm.qcow2,format=qcow2,id=hd0 `
-device virtio-blk-device,drive=hd0 `
-netdev user,id=net0 `
-device virtio-net-device,netdev=net0
```

即使你一开始安装的时候用的是4GB的内存，你后面启动的时候也可以把-m 4096 -smp 4 这一行数字改大成-m 8192 -smp 4

关闭虚拟机使用这个代码

```
poweroff
或者
shutdown -h now
```

二、系统搭建

1.代码学习

目录/文件	核心作用	你的关注点
`src/`	**最核心的源代码目录**	**这是你主要的研究战场。**内核的核心模块（如调度、内存管理、IPC）应该都在这里。
`demos/`	示例程序	展示了如何使用内核功能，是你理解API用法的好参考。
`testsuites/`	测试套件	包含了内核的单元测试和功能测试，可以帮你理解模块的预期行为。
`doc/`	**文档目录**	**请务必首先阅读这里的文档**，特别是设计文档、移植指南等。
`platform/`	平台相关代码	包含了针对不同硬件平台（如鲲鹏、QEMU virt）的适配代码。
`build/` 和 `build.py`	构建系统	编译内核的脚本和配置。你需要阅读 `README.md` 和 `doc/` 来学习如何编译。
`README.md`	**项目总览**	项目简介、快速开始指南。**请务必先** `cat README.md`。

下面我们要确定研究切入点

- **进程/线程调度** -> 关注 `src/kernel/sched/`
- **内存管理** -> 关注 `src/kernel/mm/`
- **进程间通信** -> 关注 `src/kernel/ipc/`
- **同步机制（锁、信号量）** -> 关注 `src/kernel/lock/` 或类似目录

三、开发与创新

四、测试验证